

Développement avec IA

La mise en place du prompt est la phase la plus importante de cette partie. En effet un bon prompt donne un résultat fiable et nécessite peu de corrections ensuite. Plus il est de qualité et moins il y' aura de corrections à effectuer. C'est dans cet état d'esprit que nous avons réalisé le prompt le plus efficient possible.

Prompt de base envoyé aux quatres intelligences artificielles:

Tu es un expert en Algorithmie Python Objectif : Tu dois concevoir un script du Diagramme de Voronoï qui prendra en compte des fichiers .txt qui vas renvoyer un PNG ou un SVG, un GUI conviviale et esthétique sera également disponible pour choisir quel fichier on veut et quel type de fichier on veut, ou partir de points manuels. L'affichage se fera dans le GUI également, et on devra aussi valider et faire des tests unitaires pour être sur que le voronoi fonctionne comme il le faut, si on à bien le résultat qu'il faut à chaque fois, sur cette partie tu peux utiliser une bibliothèque qui vas calculer voronoi correctement et comparer. IMPORTANT, ne pas utiliser de bibliothèque qui vas faire VORONOI directement dans le script principal, tu peux utiliser math, et les bibliothèques pour PNG/SVG etc mais pas de bibliothèque qui vas t'aider sur Voronoi Tu devra respecter le plus haut niveau de bonnes pratiques de programmation et inclure des test Le fichier .txt se constitue de cette façon : 213,247 54,424 180,29 212,237 50,370 95,26 162,300 485,174 1 ligne = Coordonnées x et Coordonnées y Donne moi tout ce qu'il faut pour cette application

Tableau des différentes IA utilisées

Nom du modèle :	Claude Code OPUS 4.6	Chat GPT-5	Gemini 3.1 Pro	Grok 4.20
Applications fonctionnelles	- résultat fonctionnel dès le première essai	Chat Gpt m'a donné un résultat assez solide dès le premier essai une architecte très bonne méthode de calcul, un gui représentative de qualité	Gemini à très bien compris le but et a su donner un code fonctionnel sans avoir à corriger. (Ce qui était attendu, étant le modèle le plus performant sur les Benchmark) GUI très réactif.	Grok m'a donner une bonne architecture dès la première requête et le projet était fonctionnel dès le début. La vitesse des réponse du modèle très impressionnante

Analyse des corrections nécessaires	- aucunes modifications à apporter	une petite correction était nécessaire pour avoir un voronoï fonctionnelle dans le dessin des formes c'était de remplacer ca <code>return mid, Point(-dx, dy)</code> par ca <code>return mid, Point(dx, dy)</code> t qui donnait un vrai résultat voronoï	Aucune correction nécessaire.	-pas de correction apporté
Comparaison IA vs développement humain	L'IA : elle a utilisé une architecture complexe plus facile à maintenir . Les responsabilités sont partagées entre le cerveau, l'interface graphique et la communication. Et dans chacun de ces dossiers il y'a des fichiers précis qui ont des fonctionnalités différentes. Humain: utilise le script simple n'a forcément pas une architecture complète pour gérer la maintenance de son code. De plus, le code n'est pas finalisé. Tous se passe dans en 3 fichiers, celui qui contient les points en txt, celui qui contient le diagramme	L'IA : a utilisé des packages une architecture maintenable et des codes plus performants et solide en utilisant l'intersection des demi plans dans mon cas qui est une notion finalement compréhensible d'un point de vue humain nous ayant fait des maths Humain : utilise le script simple n'a forcément pas une architecture aussi complète aussi décompenser pour gérer la maintenabilité de son code dans notre phase 1	Humain : Logique et script simplifié et non complet.. IA : Code complet et optimisé, qui est complexe (Algorithme Sutherland-Hodgman)	IA: arborescence plus claire et code mieux optimisé et moins lourd Humain: les responsabilité n'était pas assez séparé et le code n'était pas facile à lire et à maintenir en plus le résultat final n'était pas tout à fait ce qui était attendu

	renvoyé et enfin celui qui contient le code python. Donc pas de gros dossier.			
--	---	--	--	--

Synthèse critique IA

A. La dépendance et la perte de compétence

Au cours de la phase 2 nous avons rapidement constaté que l'IA codait plus vite et mieux que nous. On a pris des jours à se documenter, à mettre en place l'architecture du projet et nous avons fait plusieurs erreurs avant d'obtenir un projet fonctionnel. Mais cela ne veut pas dire que c'était du temps perdu. En faisant des erreurs, en cherchant des solutions nous sommes montés en compétence. Le problème avec l'IA ce n'est pas son utilisation c'est la manière dont les gens (développeurs, étudiants...) l'utilisent. Lorsqu'on se contente de copier coller les consignes à l'IA pour qu'il développe le projet, il le fait plutôt bien. Le projet est fonctionnel, l'architecture a l'air correcte et on pense avoir fait une bonne affaire puisqu'on gagne du temps et on fournit peu d'efforts. En réalité ce n'est pas une si bonne affaire puisque généralement on ne comprend pas tout le code, on a souvent du mal à le réécrire ou à l'expliquer et surtout à le débbugger. C'est un aspect important parce que dans cette ère où l'IA explose, son utilisation dans le milieu académique où on a du mal à la contrôler peut créer de mauvaises habitudes. C'est mauvais pour les futurs développeurs qui, à terme, ne comprendront peut-être même plus ce qu'ils produiront.

B. Responsabilité juridique et sécurité

Comme l'IA a été entraînée sur des millions de lignes de code, elle peut générer des morceaux de code protégés par copyright. C'est important de prendre cela en compte car on peut être poursuivi par le propriétaire original qui peut demander des dommages et intérêts. De plus si on génère tout notre code avec l'IA ça peut poser des problèmes de sécurité. Comme l'IA ne connaît pas toujours les dernières failles de sécurité, elle peut par exemple importer une bibliothèque qui contient certaines failles et on peut passer outre car on se dit que le code est fonctionnel mais on laisse sans le savoir une porte d'entrée à des pirates informatiques ou des personnes malveillantes. C'est très grave surtout si on travaille en entreprise et que le code est destiné à un projet (application ou logiciel).

C. L'efficacité au détriment de l'environnement

En quelques secondes nous avons vu que l'IA était capable de nous fournir 4 versions fonctionnelles. Le problème c'est que cela pousse à ne plus optimiser le code et à faire beaucoup de requêtes énergivores pour des petits ajustements. Cela a un impact direct sur l'environnement car chaque requête demande une quantité massive d'énergie (eau, électricité). Pour se faire un ordre d'idée, une requête Chat GPT consomme 10 fois plus d'électricité qu'une recherche Google classique. Cela soulève beaucoup de questions surtout que de nos jours on cherche de plus en plus à réduire notre impact environnemental

et à préserver l'écologie. Le gain de productivité vaut-il vraiment le coup quand on connaît les conséquences qu'il entraîne ?

Conclusion

Pour conclure, nous avons compris que l'objectif de ce projet était de nous sensibiliser à l'utilisation de l'intelligence artificielle. C'est un outil puissant mais il doit être utilisé intelligemment et avec précaution. Cela veut dire qu'il ne faut pas faire du copier coller bêtement mais toujours chercher à comprendre ce qu'on fait. Dans un monde où l'intelligence artificielle prend de plus en plus de place, et où la plupart des métiers sont menacés par cette dernière. Seuls trois éléments vont nous démarquer en tant que futurs professionnels : nos compétences techniques, notre capacité à analyser et à proposer des solutions fiables. À notre époque, être un développeur ce n'est pas juste produire du code mais aussi savoir l'optimiser et utiliser des outils respectueux de l'environnement.